



A CSR initiative by



kotak
Kotak Mahindra Bank

Kotak IISc AI-ML Centre



TANUH
Translational Artificial Intelligence
for Networked Universal Healthcare

Deep Foundation Models

Lecture Series - Part I

Sriram Ganapathy

Electrical Engineering, IISc.

Scope

- ✓ In this lecture series - what is covered
 - * The question of “what” in the topics listed
 - * A part of the “why” as well
 - * And a small part of “where” it is used
- ✓ In this lecture series - what is not covered
 - * Mathematical derivations
 - * Details of various methods, analysis and understanding
 - * Implementation aspects

Foundation models

ChatGPT definition

Foundation models are large-scale machine learning models trained on massive and diverse datasets that can be **adapted (or fine-tuned)** for a wide range of downstream tasks. They form the foundation upon which many specialized applications are built.

Examples: GPT (text), Whisper (audio), DINO (images) ...

✓ Atomic units of “most” foundation models

* Transformers

Contents - This Lecture

✓ 1.1 Transformer

- * 1.1.1 Self-attention

- * 1.1.2 encoder-decoder attention

✓ 1.2 In-context learning, low-rank adaptation

✓ 1.3 Self-supervised learning: objectives and loss functions

- * 1.3.1 Masked language modeling

Contents - This Lecture

✓ 1.1 Transformer

- * 1.1.1 Self-attention

- * 1.1.2 encoder-decoder attention

✓ 1.2 In-context learning, low-rank adaptation

✓ 1.3 Self-supervised learning: objectives and loss functions

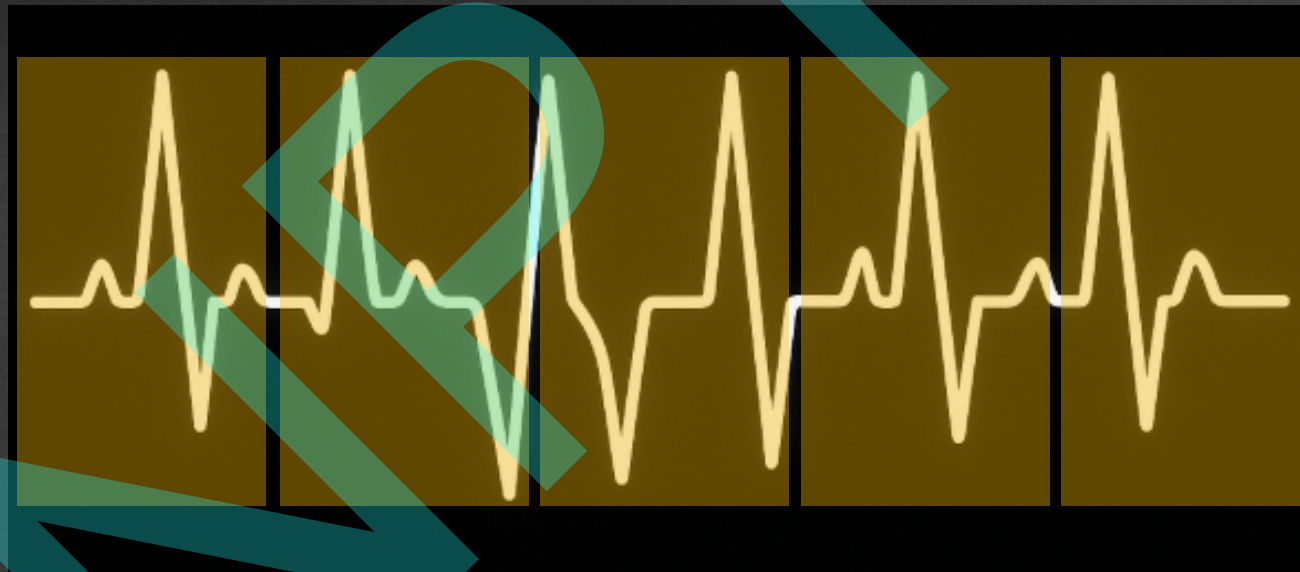
- * 1.3.1 Masked language modeling

1.1 Sequence modeling

✓ Modeling time-series data

* Input is of the form $\mathbf{x}(t)$ and output is of the form $\mathbf{y}(t)$

* Let us start with an example



$\mathbf{x}(1)$

$\mathbf{x}(2)$

$\mathbf{x}(3)$

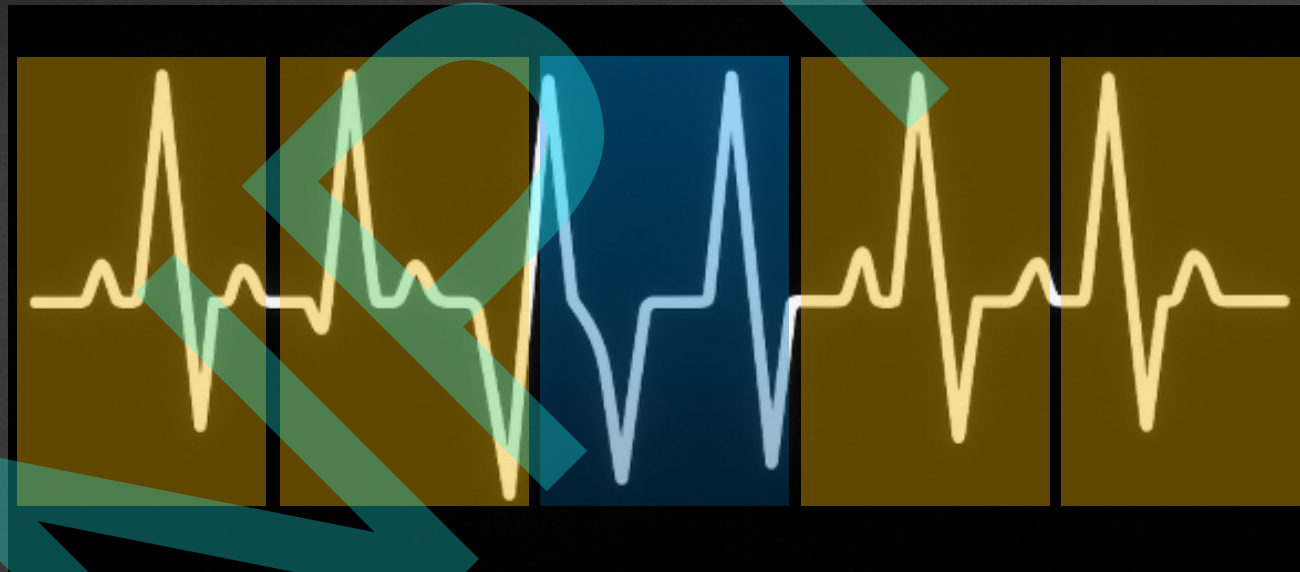
* $\mathbf{x}(t) \in \mathcal{R}^D$ denote some features or representations of windowed data.

1.1 Sequence modeling

✓ Modeling time-series data

* Input is of the form $\mathbf{x}(t)$ and output is of the form $\mathbf{y}(t)$

* Let us start with an example

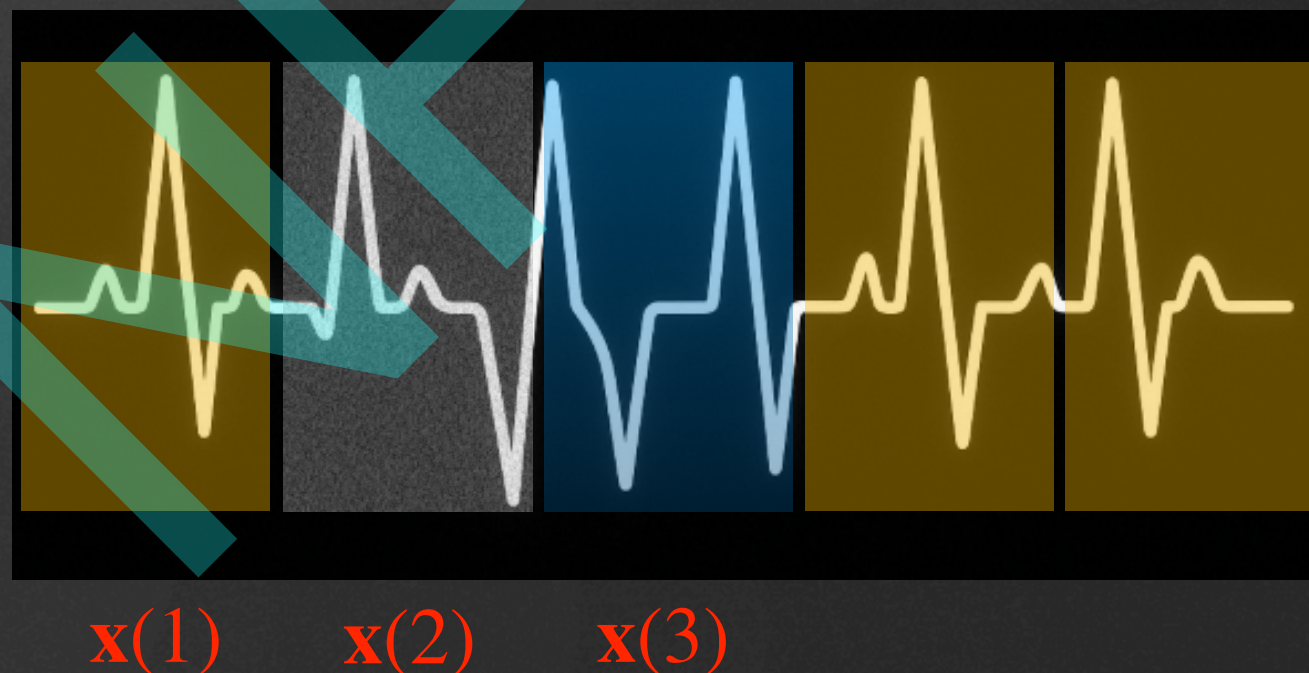


$\mathbf{x}(1)$ $\mathbf{x}(2)$ $\mathbf{x}(3)$

* $\mathbf{x}(t) \in \mathcal{R}^D$ with $\mathbf{y}(t) \in \{0,1\}$ denoting normal or abnormal heartbeat, respectively.

Sequence modeling

- ✓ Here $\mathbf{x}(t)$ and $\mathbf{y}(t)$ are ordered sequences
- * Modeling task is to predict the $\mathbf{y}(t)$ given $\mathbf{x}(t)$
- * Information needed to predict $\mathbf{y}(3)$ as abnormal is also available in other time instances.
- * Hence, a direct mapping of $\mathbf{x}(t) \rightarrow \mathbf{y}(t)$ using DNN/CNN will ignore the time dependencies.



Sequence modeling

- ✓ Here $\mathbf{x}(t)$ and $\mathbf{y}(t)$ are ordered sequences
 - * Modeling task is to predict the $\mathbf{y}(t)$ given $\mathbf{x}(t)$
 - * Information needed to predict $\mathbf{y}(3)$ as abnormal is also available in other time instances.
 - * Hence, a direct mapping of $\mathbf{x}(t) \rightarrow \mathbf{y}(t)$ using DNN/CNN will ignore the time dependencies.
- Motivates the need for sequence models
 - ✦ Sequence model $[\mathbf{x}(1), \mathbf{x}(2), \dots, \mathbf{x}(T)] \rightarrow [\mathbf{y}(1), \mathbf{y}(2), \dots, \mathbf{y}(T)]$
 - ✦ Need to model the temporal dependency between input (and output) sequences.

Sequence modeling

- ✓ Several approaches exist to model sequences
 - * Recurrent neural networks (RNN) were the most popular choice
 - * Model the hidden layers in recurrent fashion
 - * If $\mathbf{h}(t)$ is a hidden layer output, RNNs model a one-step dependency, i.e. $\mathbf{h}(t) = g(\mathbf{h}(t-1), \mathbf{x}(t))$.
 - * Modeling one-step dependency may not be the most effective framework.
 - * May suffer from vanishing gradients (memory may be limited to local context only).
- * Can the temporal dependency be modeled across multiple time-instances jointly ... a.k.a. attention

Attention is all you need

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Transformers

✓ Key building blocks

- * Attention, Residual connection, normalization, feed-forward layers and positional encoding

✓ Big picture

- * Every input representation “attends” to every other input representation including itself.
- * Generated a weighted combination of all representations, where the weights are based on the similarity between representations.

Self-attention

- ✓ Assume that words are represented as vectors in some embedding space (for example, word2vec). Let $\mathbf{x}_t \in \mathcal{R}^{1 \times D}$

Word
Embeddings

I am a student of machine learning

$\mathbf{x}_1$

$\mathbf{x}_2$

$\mathbf{x}_3$

$\mathbf{x}_4$

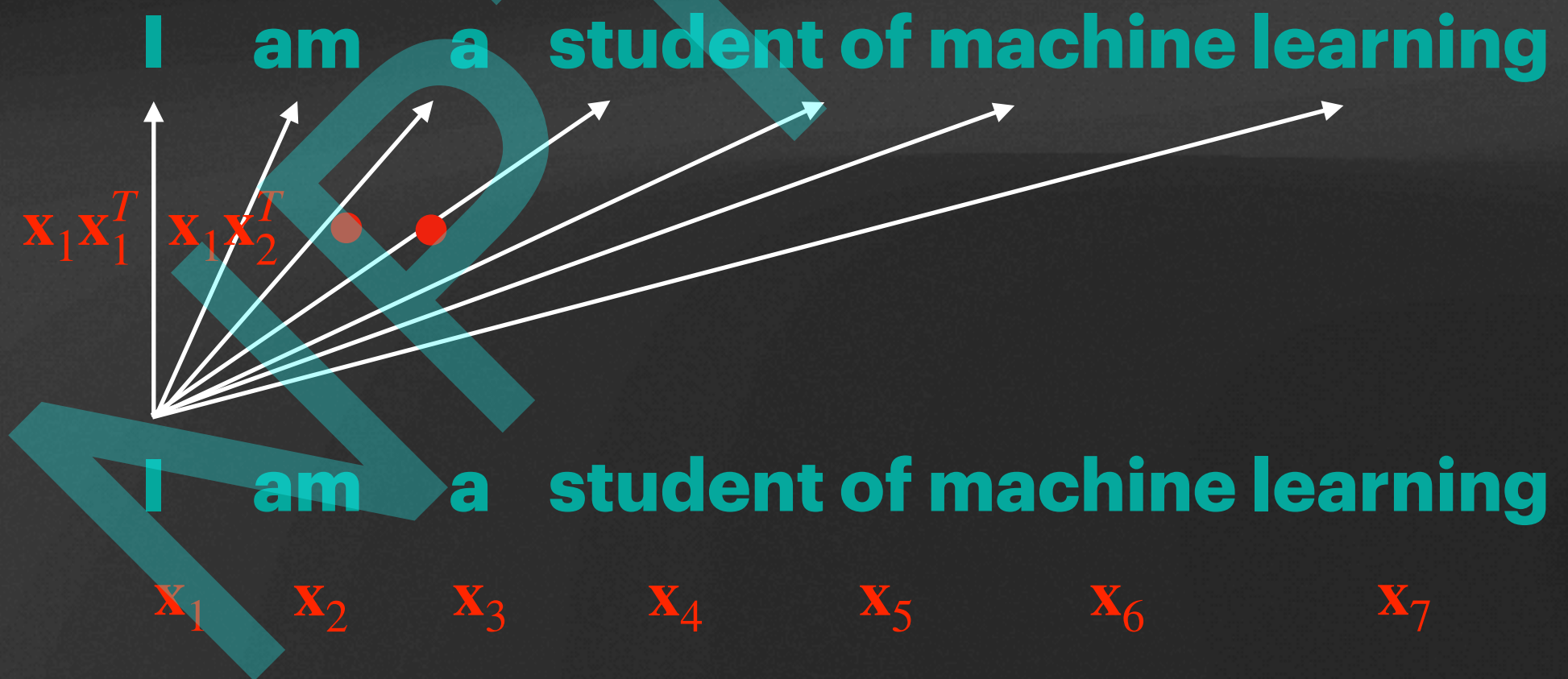
$\mathbf{x}_5$

$\mathbf{x}_6$

$\mathbf{x}_7$

Self-attention

- ✓ Compute the cross-correlation of every vector with every other vector in the input



Self-attention

✓ Weigh every vector with the weights from the cross-correlation.

$\mathbf{x}_1 \mathbf{x}_1^T$	$\mathbf{x}_1 \mathbf{x}_2^T$					

With

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \dots \\ \mathbf{x}_T \end{bmatrix} \in \mathcal{R}^{T \times D}$$

$$\mathbf{A} = \mathbf{X} \mathbf{X}^T \in \mathcal{R}^{T \times T}$$

$$\mathbf{E} = \mathbf{A} \mathbf{X} \in \mathcal{R}^{T \times D}$$

Cross-correlation matrix captures the pairwise relationship between every pair of inputs.

Self-attention

✓ There is nothing learnable in using input directly —> Use transformed versions of the input.

k_1	$q_1 k_1^T$	$q_1 k_2^T$				
k_2						
k_3						
	q_1	q_2	q_3			

With

$$X = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \dots \\ \mathbf{x}_T \end{bmatrix} \in \mathcal{R}^{T \times D}$$

$$Q = XW_Q \in \mathcal{R}^{T \times d}$$

$$K = XW_K \in \mathcal{R}^{T \times d}$$

$$V = XW_V \in \mathcal{R}^{T \times d}$$

$$A = QK^T \in \mathcal{R}^{T \times T}$$

Dot-product captures pair-wise relationships

Self-attention

✓ There is nothing learnable in using input directly —> Use transformed versions of the input.

$\mathbf{k}_1$	$\mathbf{q}_1 \mathbf{k}_1^T$	$\mathbf{q}_1 \mathbf{k}_2^T$				
$\mathbf{k}_2$						
$\mathbf{k}_3$						
	$\mathbf{q}_1$	$\mathbf{q}_2$	$\mathbf{q}_3$			

With

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \dots \\ \mathbf{x}_T \end{bmatrix} \in \mathcal{R}^{T \times D}$$

$$\mathbf{Q} = \mathbf{X} \mathbf{W}_{\mathbf{Q}} \in \mathcal{R}^{T \times d}$$

$$\mathbf{K} = \mathbf{X} \mathbf{W}_{\mathbf{K}} \in \mathcal{R}^{T \times d}$$

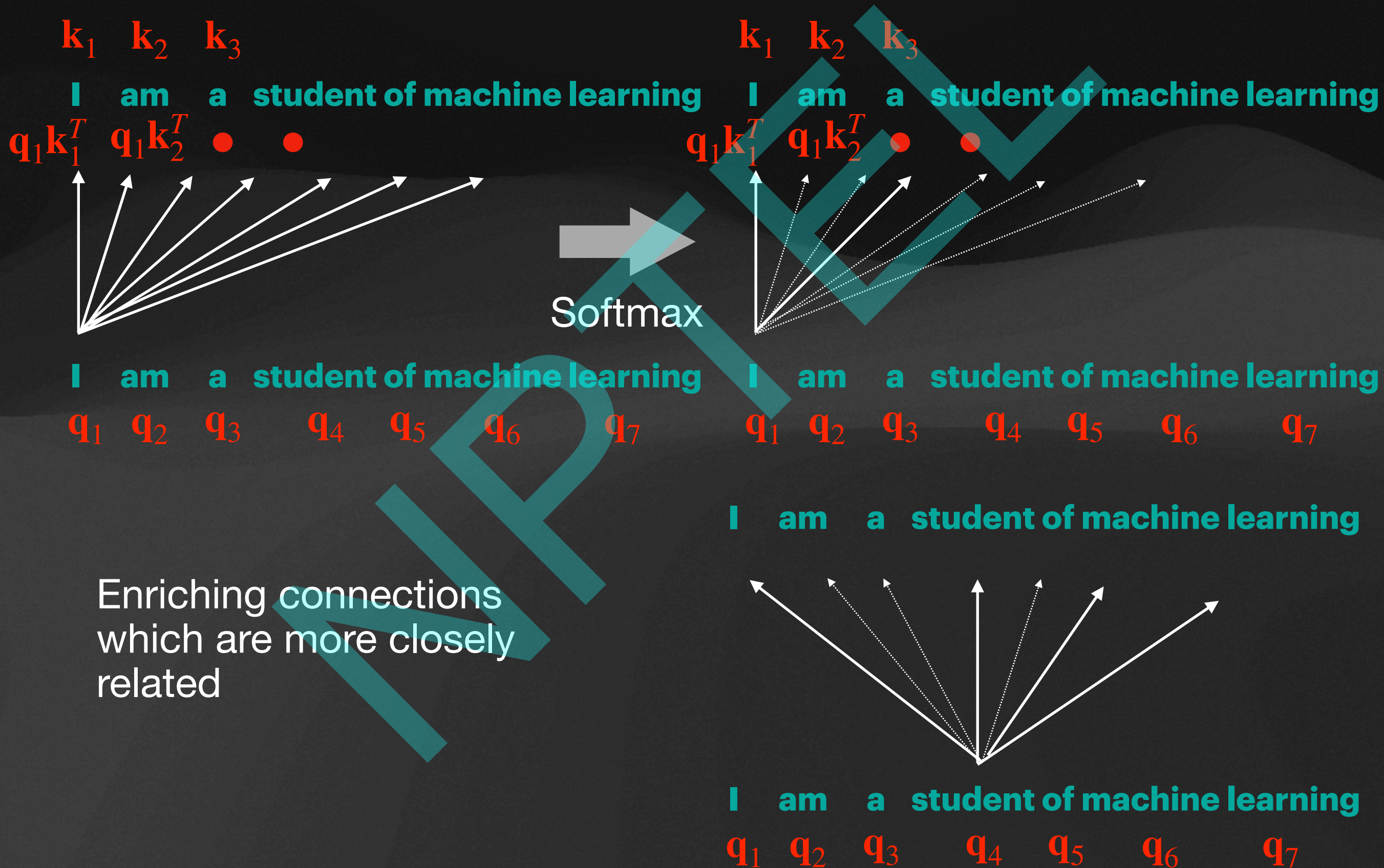
$$\mathbf{V} = \mathbf{X} \mathbf{W}_{\mathbf{V}} \in \mathcal{R}^{T \times d}$$

$$\mathbf{A} = \mathbf{Q} \mathbf{K}^T \in \mathcal{R}^{T \times T}$$

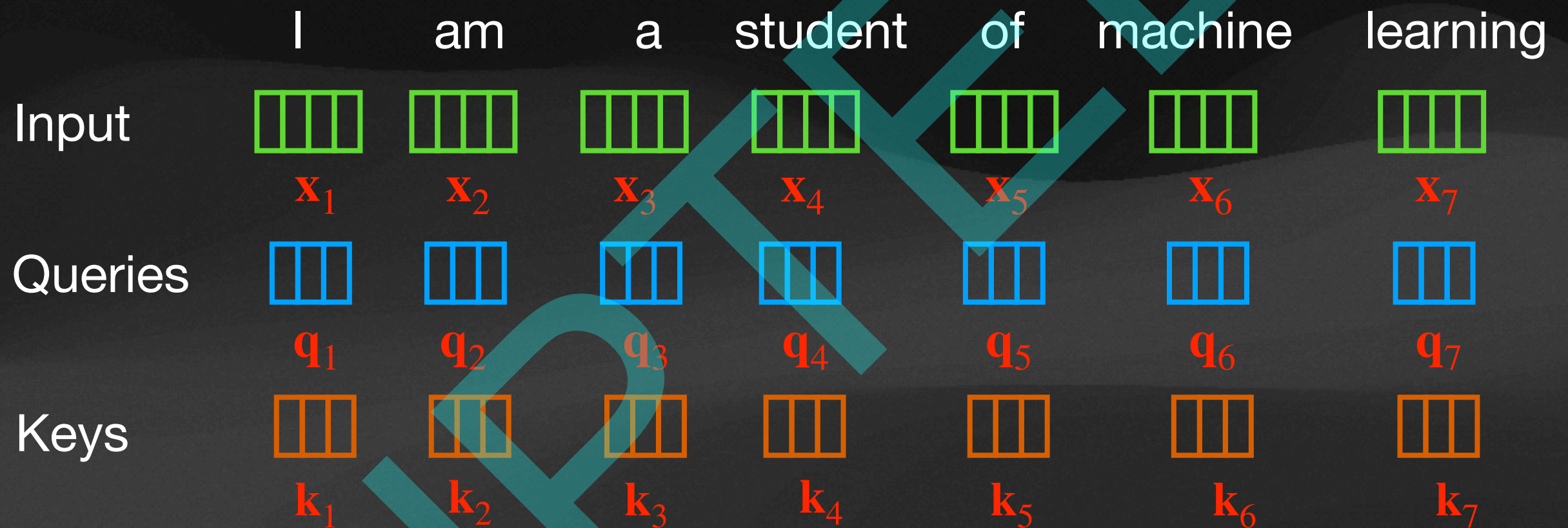
$$\mathbf{E} = \text{softmax}\left(\frac{\mathbf{A}}{d}\right) \mathbf{V} \in \mathcal{R}^{T \times d}$$

Scaled dot-product
attention

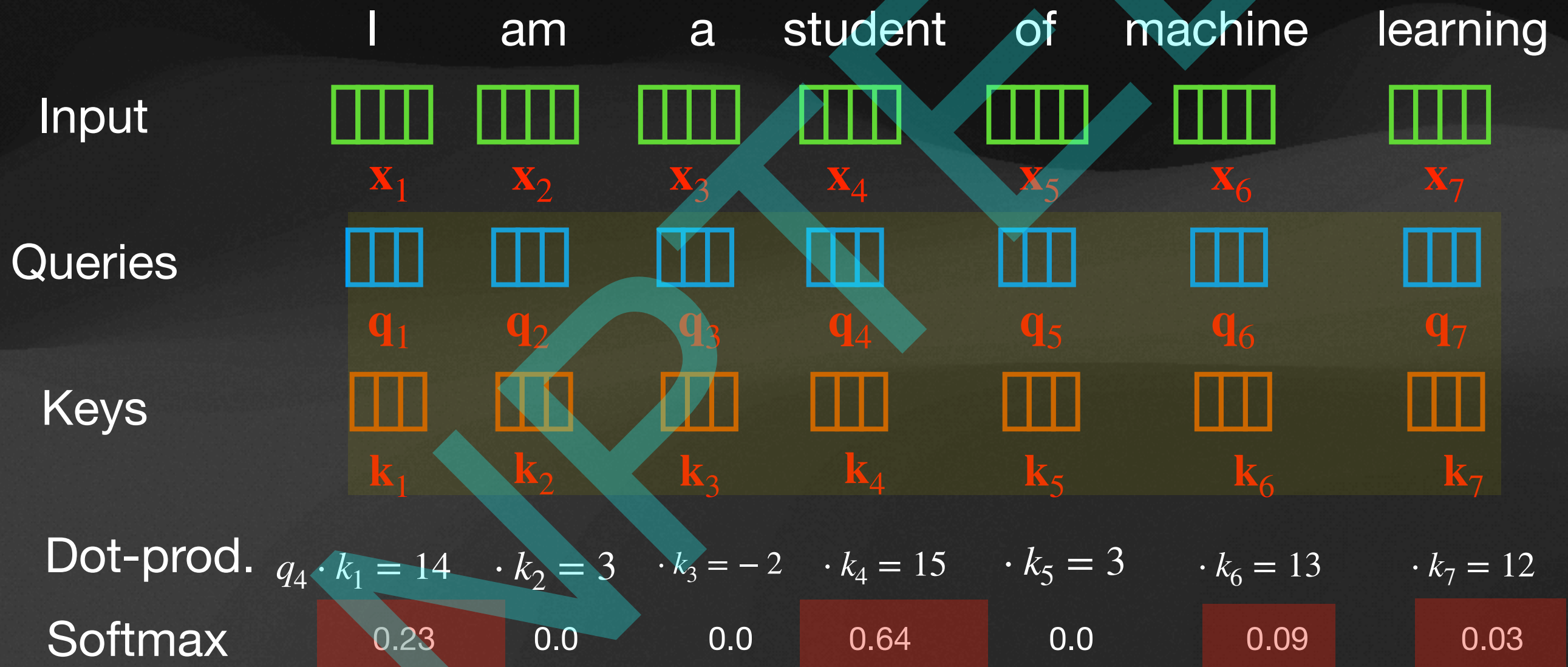
Self-attention



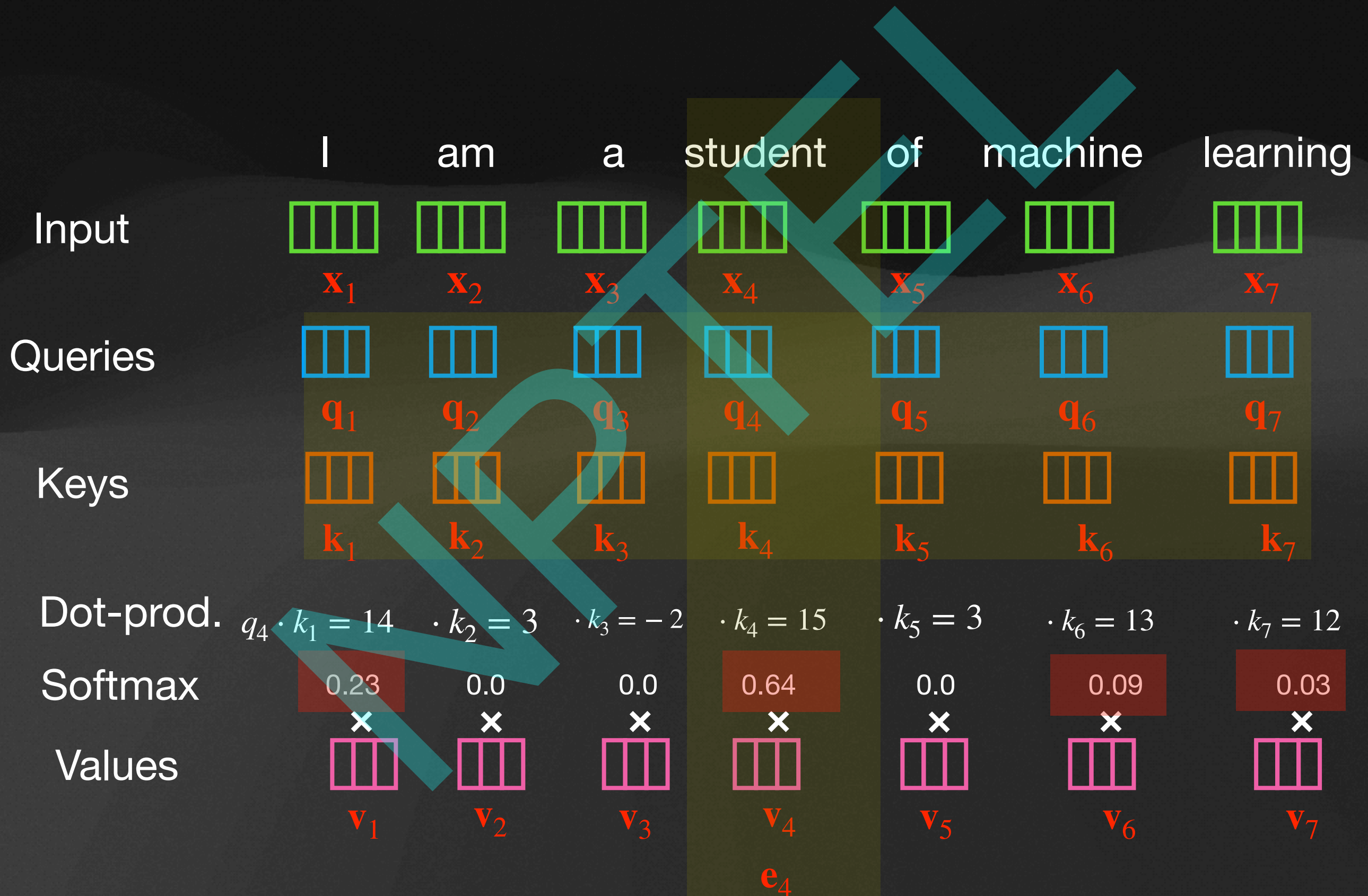
Closer look at the operations



Closer look at the operations



Closer look at the operations

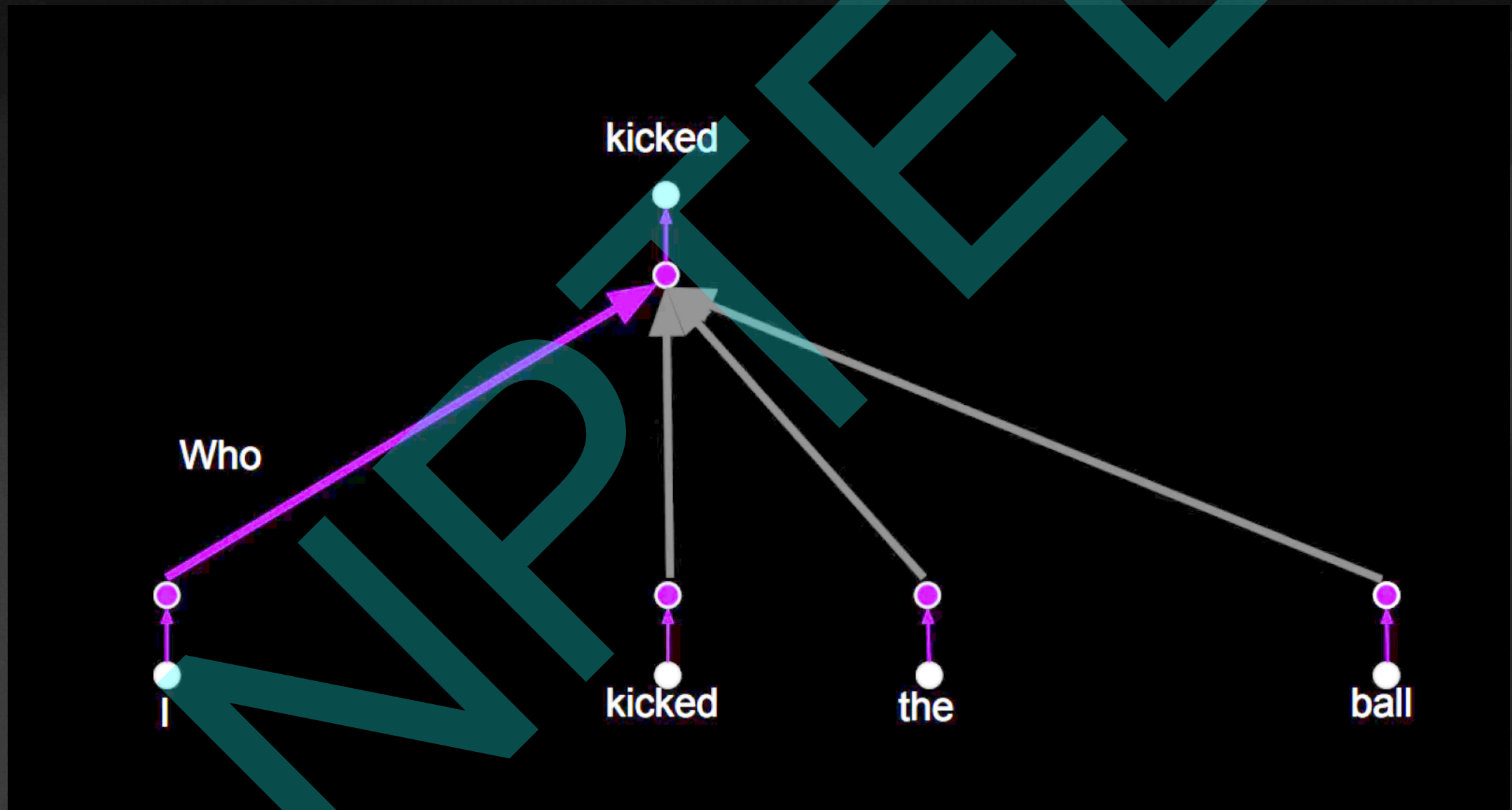


Time for some ...

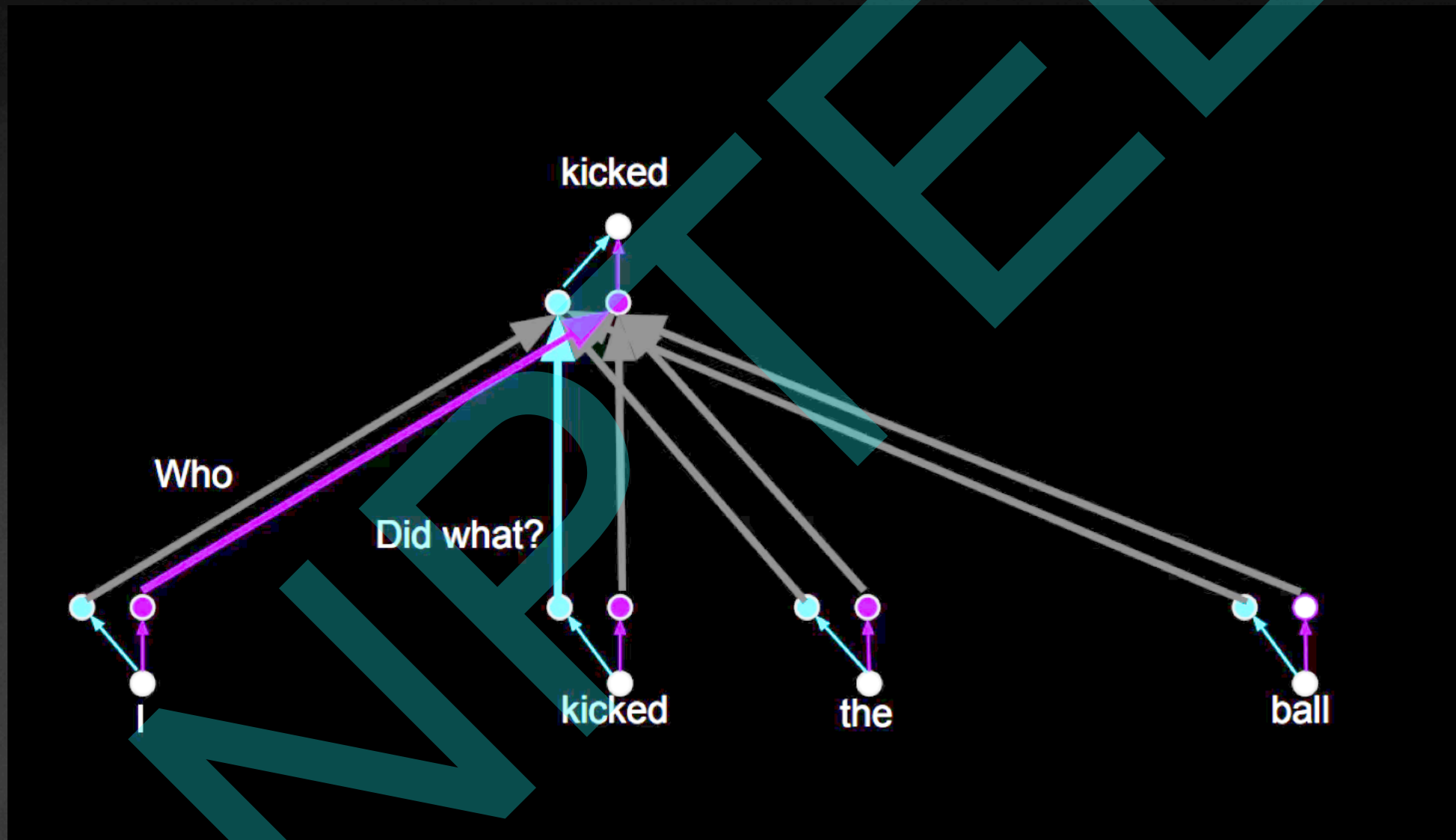
NPTF

Q&A

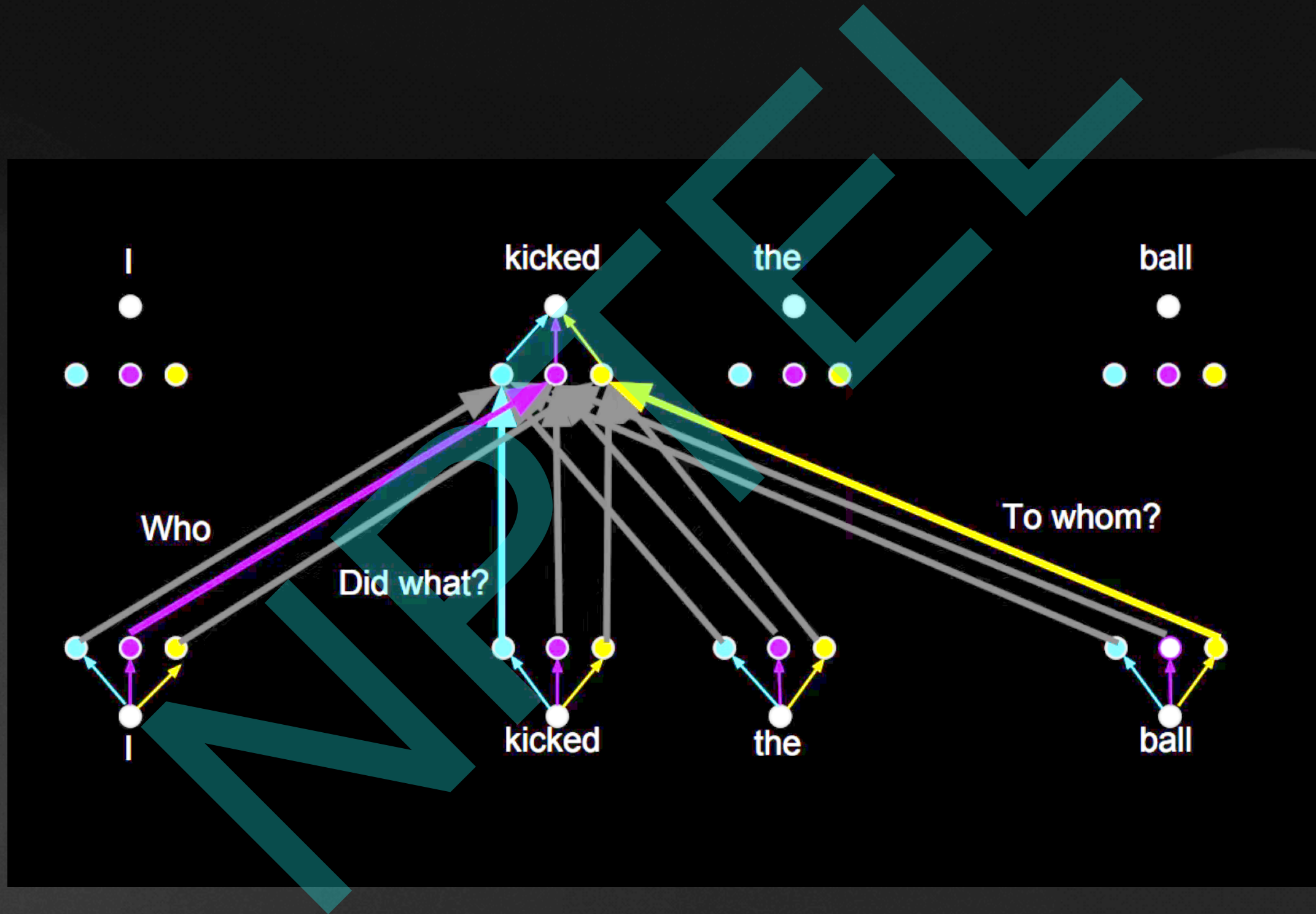
Multi-head attention



Multi-head attention



Multi-head attention



Multi-head attention

- ✓ Concatenate attention from multiple heads,

$$\mathbf{E}_k = \text{softmax}\left(\frac{\mathbf{A}_k}{d_k}\right) \mathbf{V}_k \in \mathcal{R}^{T \times d_k} \quad \mathbf{E} = [\mathbf{E}_1 \dots \mathbf{E}_h]$$

- ✓ Typically, $d = h \times d_k$
- ✓ The output of Multi-head attention goes to subsequent processing steps

Application of Transformer architecture

✓ Big picture

- * Useful as feature encoders (representation learning models)

 - Encoder models

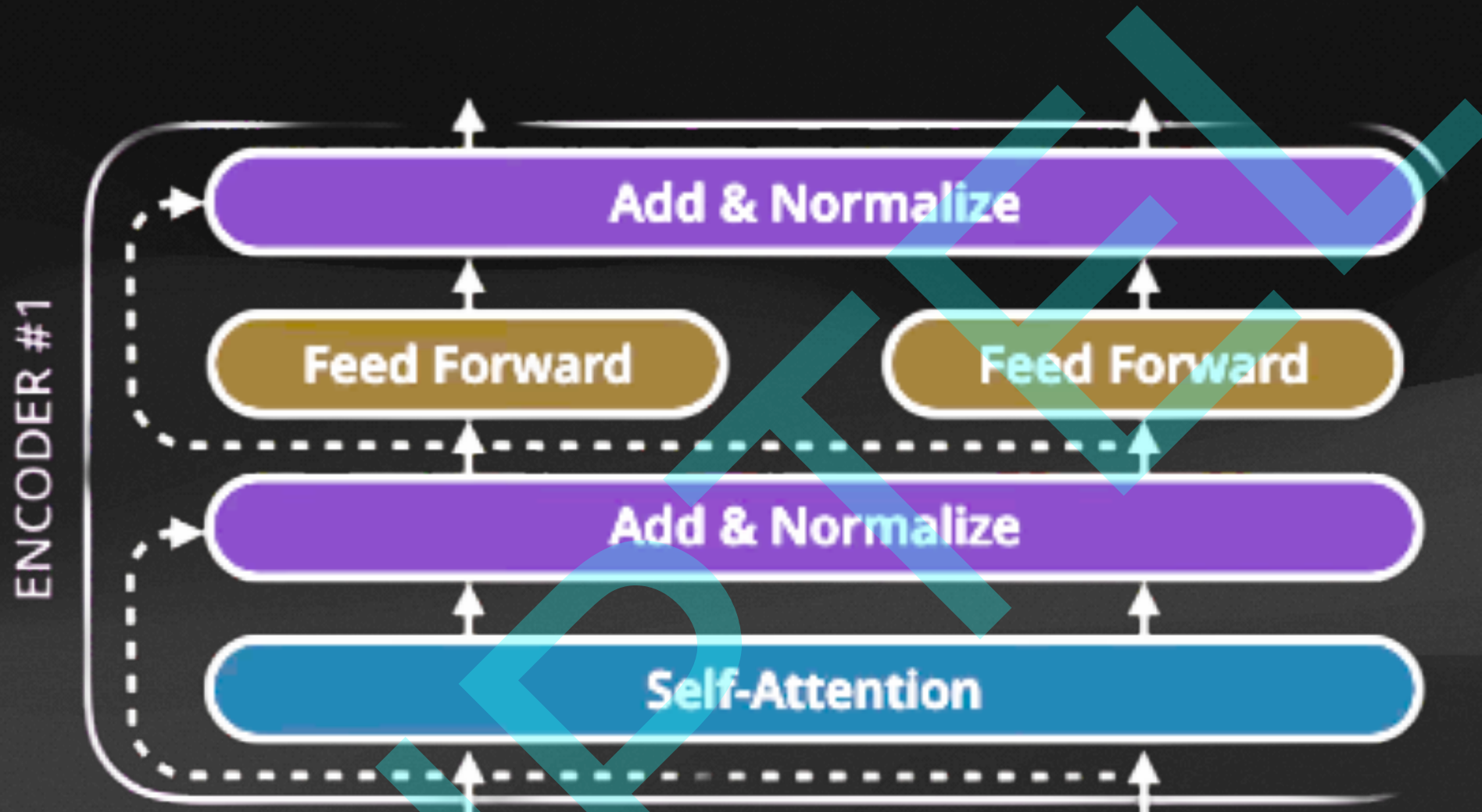
- * Applications to dedicated tasks

 - Encoder-decoder attention

- * Free-form generation

 - Decoder models

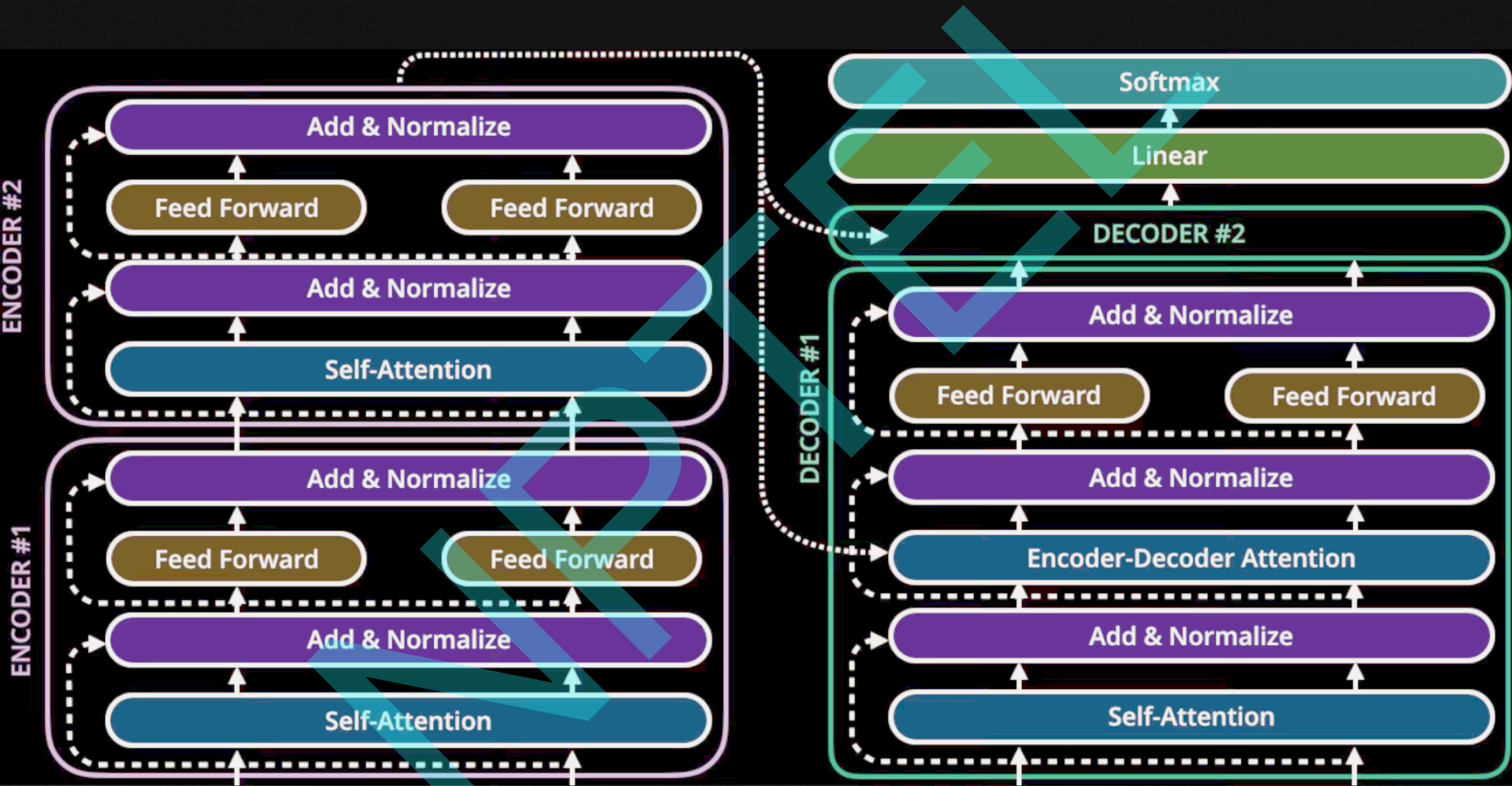
Transformer single processing layer



Layer norm - A normalization for each vector, rescales and recenters the activations across the feature dimension(s).

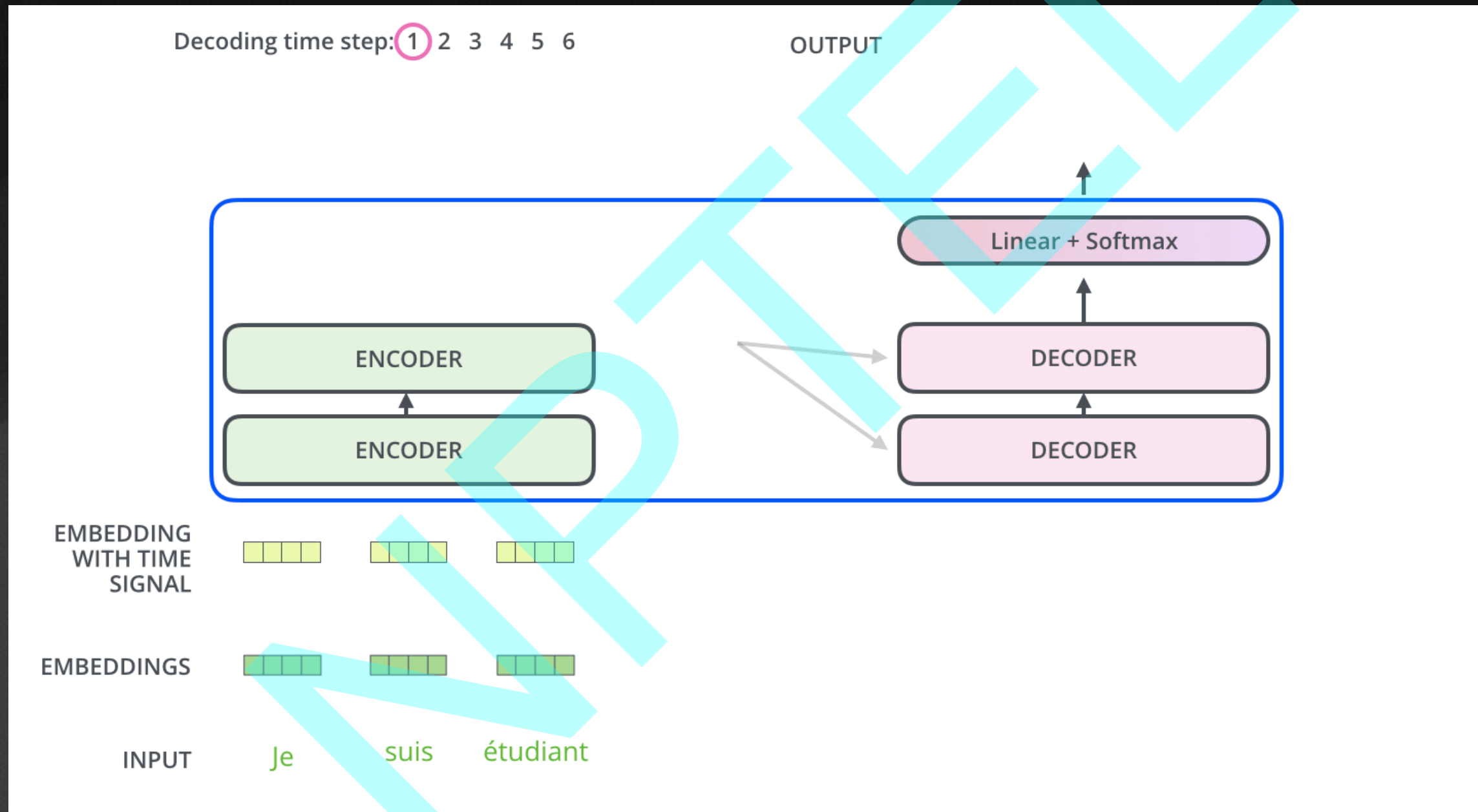
Residual connections - Increases the gradient flow and stability.

Encoder Decoder Attention



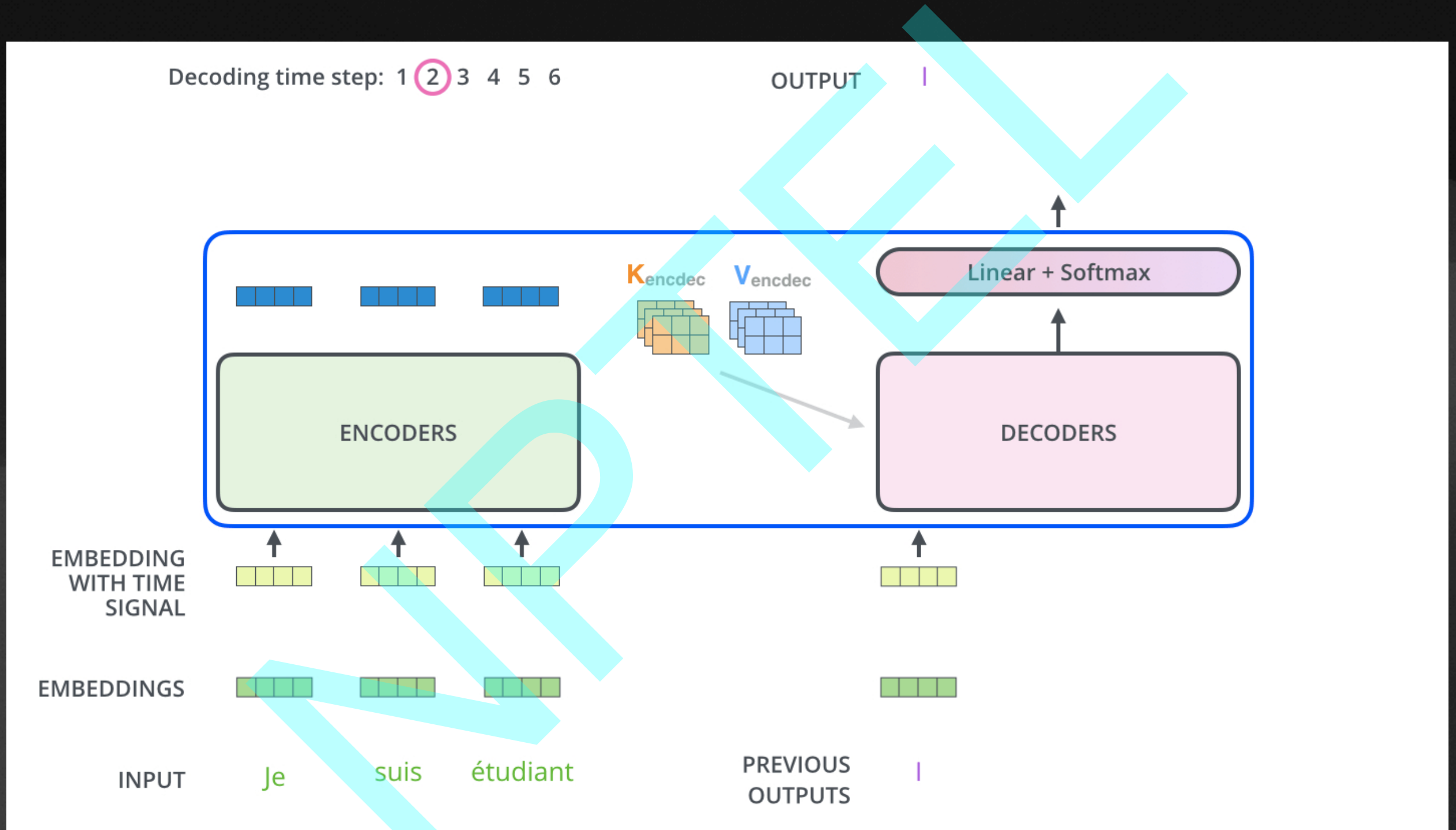
Key and values from the last encoder layer are used along with the query from the decoder

Encoder-Decoder Attention



Key and values from the last encoder layer are used along with the query from the decoder

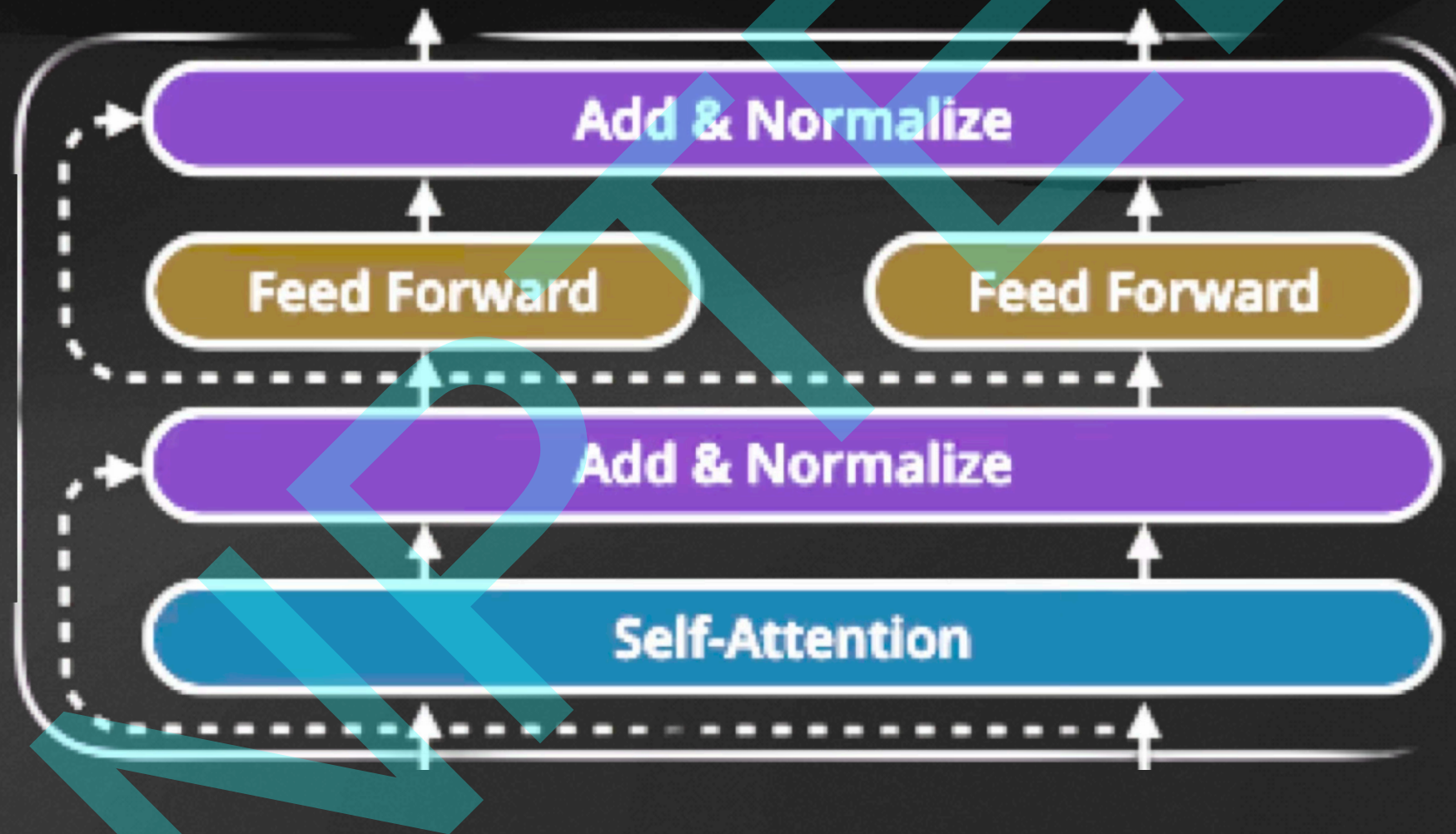
Encoder-Decoder Attention



Key and values from the last encoder layer are used along with the query from the decoder

Decoder only models

- ✓ Model contains only self-attention layers



- ✓ Difference between encoder only models and decoder only models is in the loss function

Decoder only models

- ✓ Model contains only self-attention layers
- ✓ Difference between encoder only models and decoder only models is in the loss function
- * Loss function in a decoder-only model $\mathcal{L} = - \sum_t P(\mathbf{x}_t | \mathbf{x}_{<t})$
- * Model is learned to predict the next output prediction in a sequence
- ◎ Most of the language modeling and creative generation tasks can be represented as a slot-filling task
 - For example, question answering can be considered as structured next token prediction of the form (<Question>, ...)

Positional encoding

- ✓ As the attention modeling treats all input time instances equally
 - * Order information is not available
 - * For example, without order information,
 - ◎ Next word prediction in “I went to play in the ...” is the same as “Went play I the in ...”
- ✓ In the transformer stack this is tackled by adding an embedding to the input vectors
 - * $\mathbf{x}_t \leftarrow \mathbf{x}_t + \mathbf{p}_t$ $\mathbf{p}_{t,2i} = \sin\left(\frac{t}{K^{\frac{2i}{d}}}\right)$, $\mathbf{p}_{t,2i+1} = \cos\left(\frac{t}{K^{\frac{2i}{d}}}\right)$ where i is the feature dimension and K is fixed constant (=10000).
 - * Recent efforts learnable embeddings or relative position embeddings for improved modeling.

Why are transformers successful

Advantage	Transformer Capability	Why It Matters
Parallelization	Processes all tokens simultaneously via matrix operations	Enables training on GPUs at scale, unlike RNNs which are sequential
Long-range dependency modeling	Self-attention allows each token to directly attend to all others	No vanishing gradient issues, captures global context efficiently
Scalability	Number of parameters and data can be scaled almost arbitrarily	Foundation of modern LLMs (GPT, PaLM, LLaMA)
Interpretability	Attention weights show which tokens influence which	Easier to visualize and analyze compared to hidden RNN states
Flexibility across modalities	Works for text, images, audio, video	Enables multimodal models like CLIP, GPT-4o, Gemini
Transfer learning	Same architecture can be pretrained on large corpora and fine-tuned	Paved the way for BERT, GPT, T5, etc.

Time for some ...

NPTF

Q&A

Contents - This Lecture

✓ 1.1 Transformer

- * 1.1.1 Self-attention

- * 1.1.2 encoder-decoder attention

✓ 1.2 In-context learning, low-rank adaptation

✓ 1.3 Self-supervised learning: objectives and loss functions

- * 1.3.1 Masked language modeling

Contents - This Lecture

✓ 1.1 Transformer

- * 1.1.1 Self-attention

- * 1.1.2 encoder-decoder attention

✓ 1.2 In-context learning, low-rank adaptation

✓ 1.3 Self-supervised learning: objectives and loss functions

- * 1.3.1 Masked language modeling

1.2 Deep Foundation Models

- ✓ Moving trained models for tasks and domains beyond ones seen in training
- * Model parameters frozen —> In-context learning
- * Model adaptation with only a small number of parameters —> Low-rank adaptation

In-context learning

Language Models are Few-Shot Learners

Tom B. Brown*

Benjamin Mann*

Nick Ryder*

Melanie Subbiah*

Jared Kaplan[†]

Prafulla Dhariwal

Arvind Neelakantan

Pranav Shyam

Girish Sastry

Amanda Askell

Sandhini Agarwal

Ariel Herbert-Voss

Gretchen Krueger

Tom Henighan

Rewon Child

Aditya Ramesh

Daniel M. Ziegler

Jeffrey Wu

Clemens Winter

Christopher Hesse

Mark Chen

Eric Sigler

Mateusz Litwin

Scott Gray

Benjamin Chess

Jack Clark

Christopher Berner

Sam McCandlish

Alec Radford

Ilya Sutskever

Dario Amodei

OpenAI

In-context learning

✓ Decoder only models can

* Learn a new-task purely using examples

* For example

◎ (Federer, Tennis), (Kohli, ____) ==>

- Model learns the relationship between pairs given and answers "Cricket"

◎ (Federer, Switzerland), (Ronaldo, ____) ==>

- Now the model answers as "Portugal"

In-context learning

✓ Formally

* The input is of the form

- $(\mathcal{X}_1, \mathcal{Y}_1), (\mathcal{X}_2, \mathcal{Y}_2), \dots, (\mathcal{X}_n, \mathcal{Y}_n), (\mathcal{X}_{n+1}, \dots)$
- The model learns the mapping function $f(\mathcal{X}) \rightarrow \mathcal{Y}$
 - The weights of the model are frozen
- How is it possible
 - Each transformer layer builds an internal representation of the examples in the context.
 - The self-attention mechanism allows the model to dynamically “retrieve” relevant examples when processing a new query token.
 - This leads to behavior resembling **few-shot learning** — i.e., learning from a few examples

In-context learning

✓ Types of In-context learning

Type	Description	Example
Zero-shot ICL	The model performs a task using only a natural-language instruction. (Model has not seen this task during fine-tuning)	"Translate to French: apple → ?"
One-shot ICL	The model is given one example of input-output.	"apple → pomme, banana → ?"
Few-shot ICL	The model is given multiple examples to infer a pattern.	cat → chat dog → chien house → maison tree → ?

Why does in-context learning work

Published as a conference paper at ICLR 2023

WHAT LEARNING ALGORITHM IS IN-CONTEXT LEARNING? INVESTIGATIONS WITH LINEAR MODELS

Ekin Akyürek^{1,2,a} Dale Schuurmans¹ Jacob Andreas^{*2} Tengyu Ma^{*1,3,b} Denny Zhou^{*1}

¹Google Research ²MIT CSAIL ³Stanford University ^{*}collaborative advising

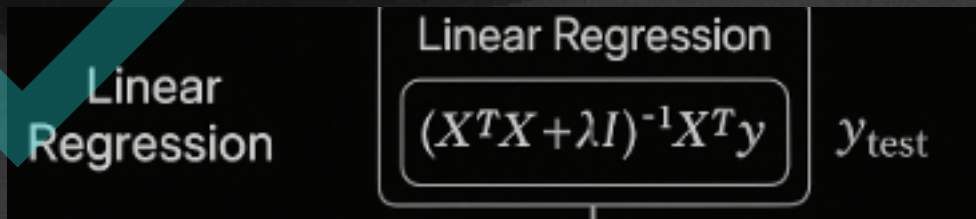
Regression-based In-Context Learning

Examples

(x_1, y_1)
 (x_2, y_2)
 $\vdots$
 (x_n, y_n)



$\tilde{y}_{\text{test}} = w^T x_{\text{test}}$
**Predicted
Test Output**



The variables needed to compute linear regression outputs can be stored in the hidden activations during the forward pass through the input itself

Low-rank adaptation

LoRA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Edward Hu* **Yelong Shen*** **Phillip Wallis** **Zeyuan Allen-Zhu**
Yuanzhi Li **Shean Wang** **Lu Wang** **Weizhu Chen**
Microsoft Corporation
{edwardhu, yeshe, phwallis, zeyuana,
yuanzhil, swang, luw, wzchen}@microsoft.com
yuanzhil@andrew.cmu.edu
(Version 2)

Low-rank adaptation

✓ Given a pre-trained transformer model with weights (for example, let $W_0 \in \mathcal{R}^{D \times d}$)

* Low-rank adaptation freezes the W_0

◎ But learns a smaller number of parameters

- $\Delta W = BA$

- $W = W_0 + \alpha \cdot \Delta W = W_0 + \alpha \cdot BA,$

- Where α is a scaling factor. $A \in \mathcal{R}^{r \times d}, B \in \mathcal{R}^{D \times r}$

- Only the matrices A, B are learned from the data

Low-rank adaptation

- ✓ Parameter saving by low-rank adaptation
 - * If one were to update the base model W_0
 - Need to learn $D \times d$ parameters
 - * Learning only A, B parameters
 - Need to learn $r(D + d)$ parameters
 - Savings for $D = 1024, d = 128, r = 8$ is
 - $8(1024 + 128)/(1024 \times 128) = 0.0703$ (7% parameters are alone updated)
- Methods like LORA fall into larger category of methods called parameter efficient fine-tuning (PEFT).

Low-rank adaptation

- ✓ LoRA is typically inserted into linear projection layers of transformer blocks:
 - * Query (Q) and Value (V) projection matrices in self-attention layers.
 - * Rarely also in feedforward networks.
- ✓ Why is it very successful — because downstream tasks often require **specific but limited shifts** in representation space, not a complete re-learning of all features.

Time for some ...

NPTF

Q&A

Contents - This Lecture

✓ 1.1 Transformer

- * 1.1.1 Self-attention

- * 1.1.2 encoder-decoder attention

✓ 1.2 In-context learning, low-rank adaptation

✓ 1.3 Self-supervised learning: objectives and loss functions

- * 1.3.1 Masked language modeling

1.3 Self-supervision

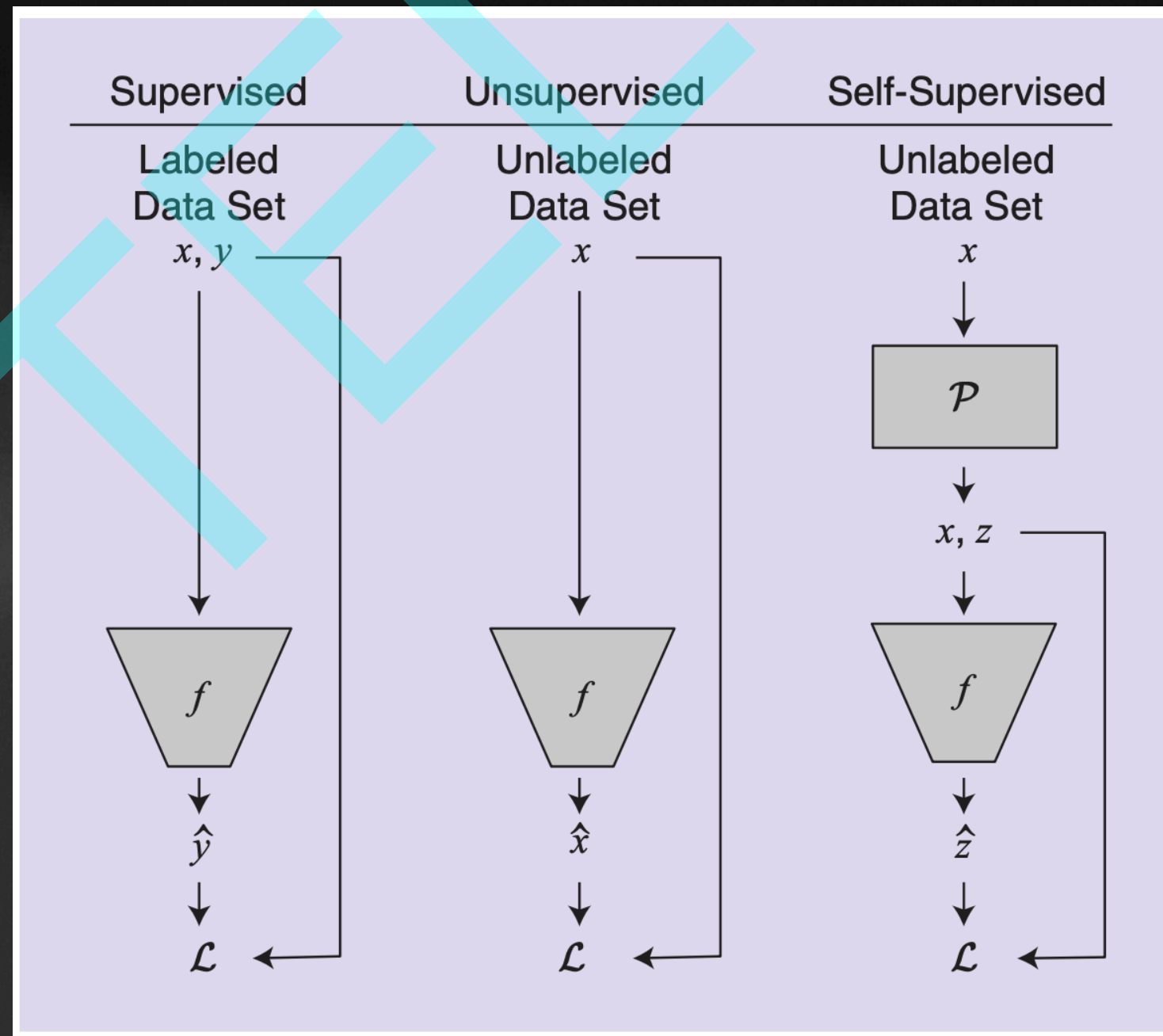
✓ Why do we need self-supervision

Data	Remark
Supervised	✓ Clear directed Goal ✗ Expensive to collect Limited Availability Task specific
Unsupervised	✓ Available in Abundance Allows Generic modeling ✗ No task or goal alignment

✓ Can we get the best of both worlds

1.3 Self supervision

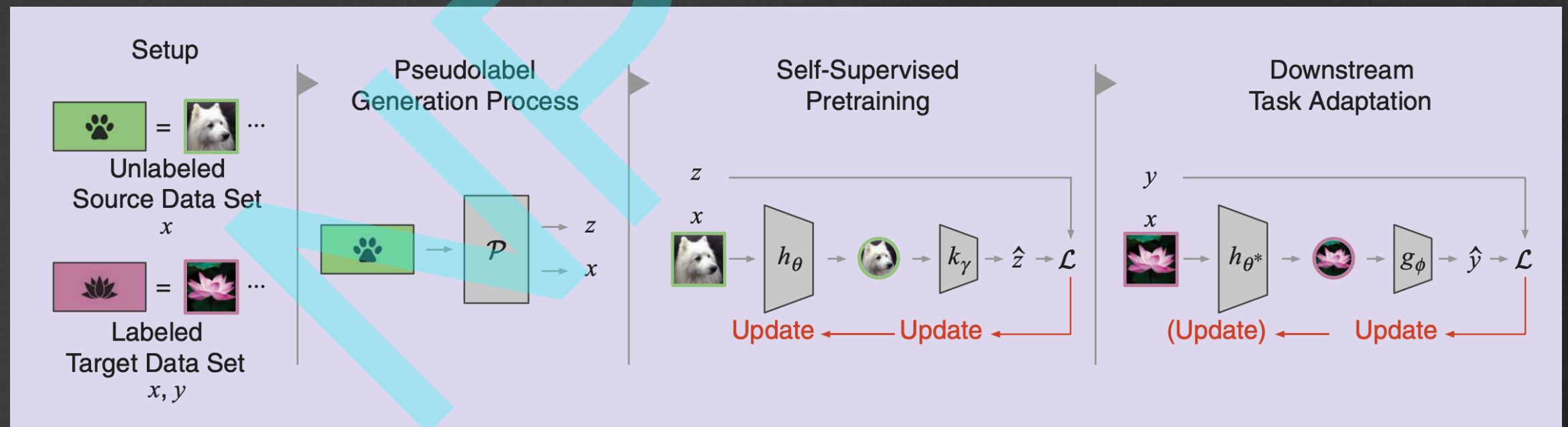
- ✓ Different from supervised and unsupervised learning
- ✓ Does not perform distribution learning or reconstruction
- ✓ Uses a pretext task
- ✓ Performing contrastive or predictive learning
- ✓ Using large volumes of unsupervised data



Ericsson, Linus, et al. "Self-supervised representation learning: Introduction, advances, and challenges." *IEEE Signal Processing Magazine* 39.3 (2022): 42-62.

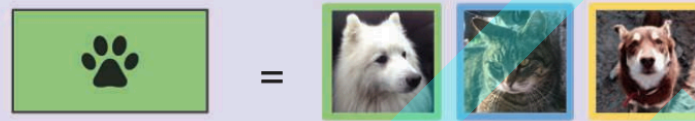
Self supervision - principle

- ✓ Two levels of modeling with unsupervised data
- ✓ Generating a pseudo-label
- ✓ Learning the upstream model
- ✓ Downstream task performs fine-tuning of the SSL model.

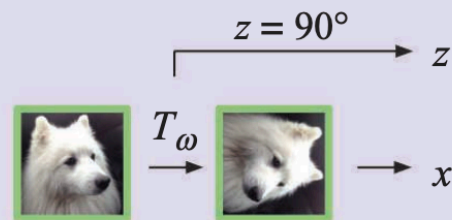


Self supervision - pre-text task

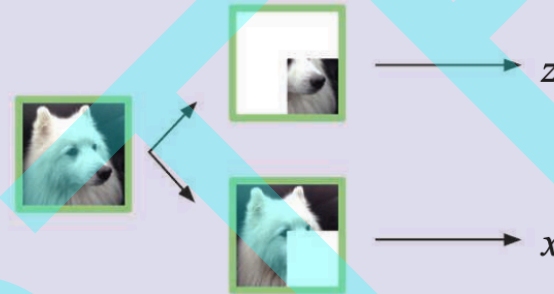
Pseudolabel Generation Processes



Transformation Prediction



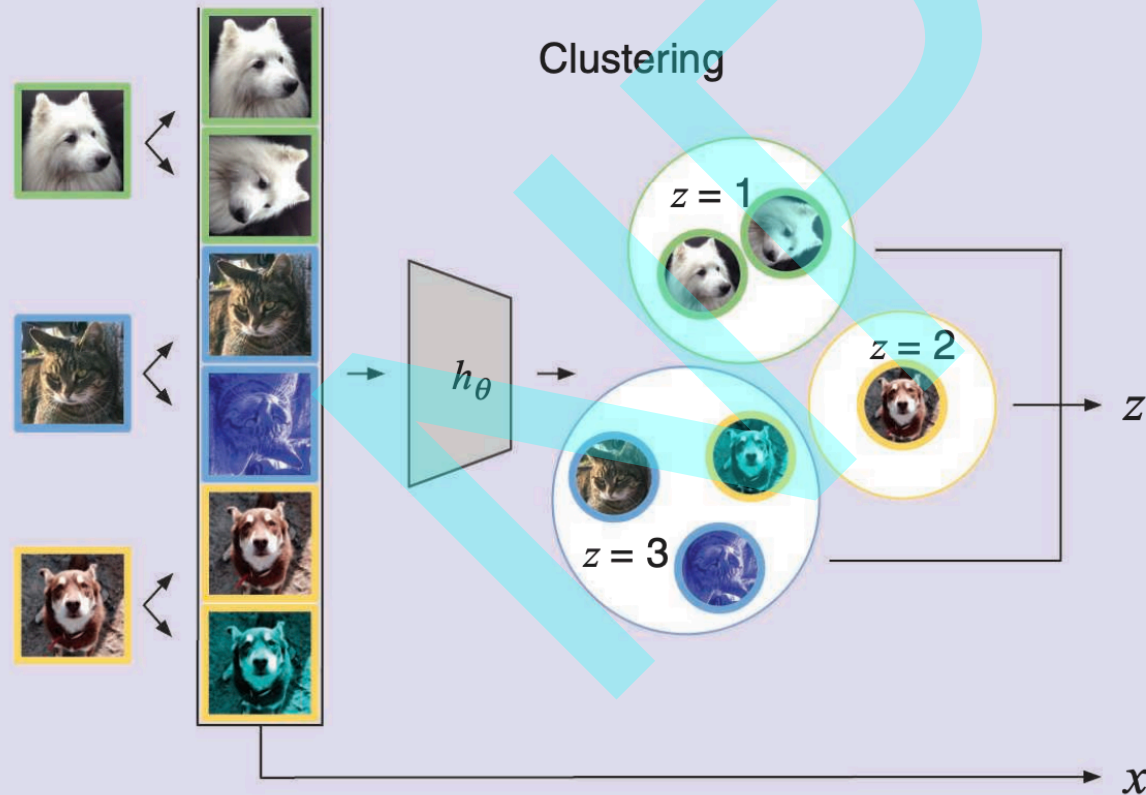
Masked Prediction



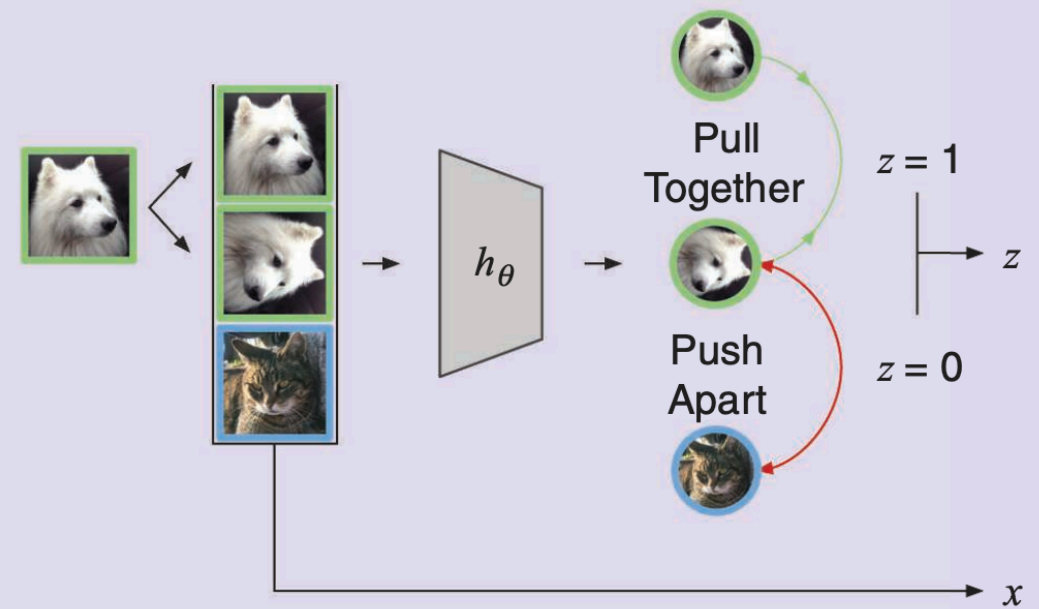
Instance Discrimination



Clustering



Contrastive Instance Discrimination



Self-supervision as a task

- ✓ Masking out portions of the input data
- * Pass the rest of the embeddings (with zeros or random entries at the masked locations) to the transformer encoder
- * Have the model predict the word tokens in the masked portions - Masked Language Modelling (MLM)

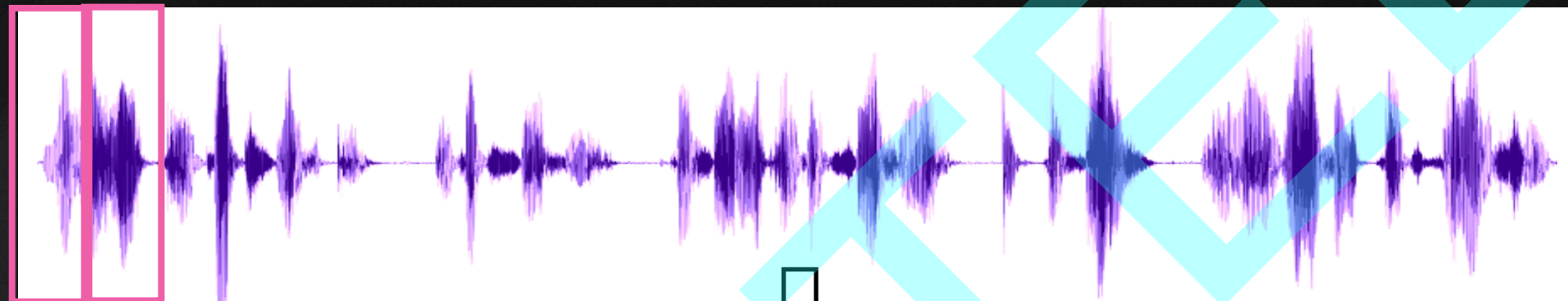
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova
Google AI Language
{jacobdevlin, mingweichang, kentonl, kristout}@google.com



Beyond text

LLMs have moved beyond text to any general discrete sequence



Audio

“Spectral or Deep” representations



Quantizer

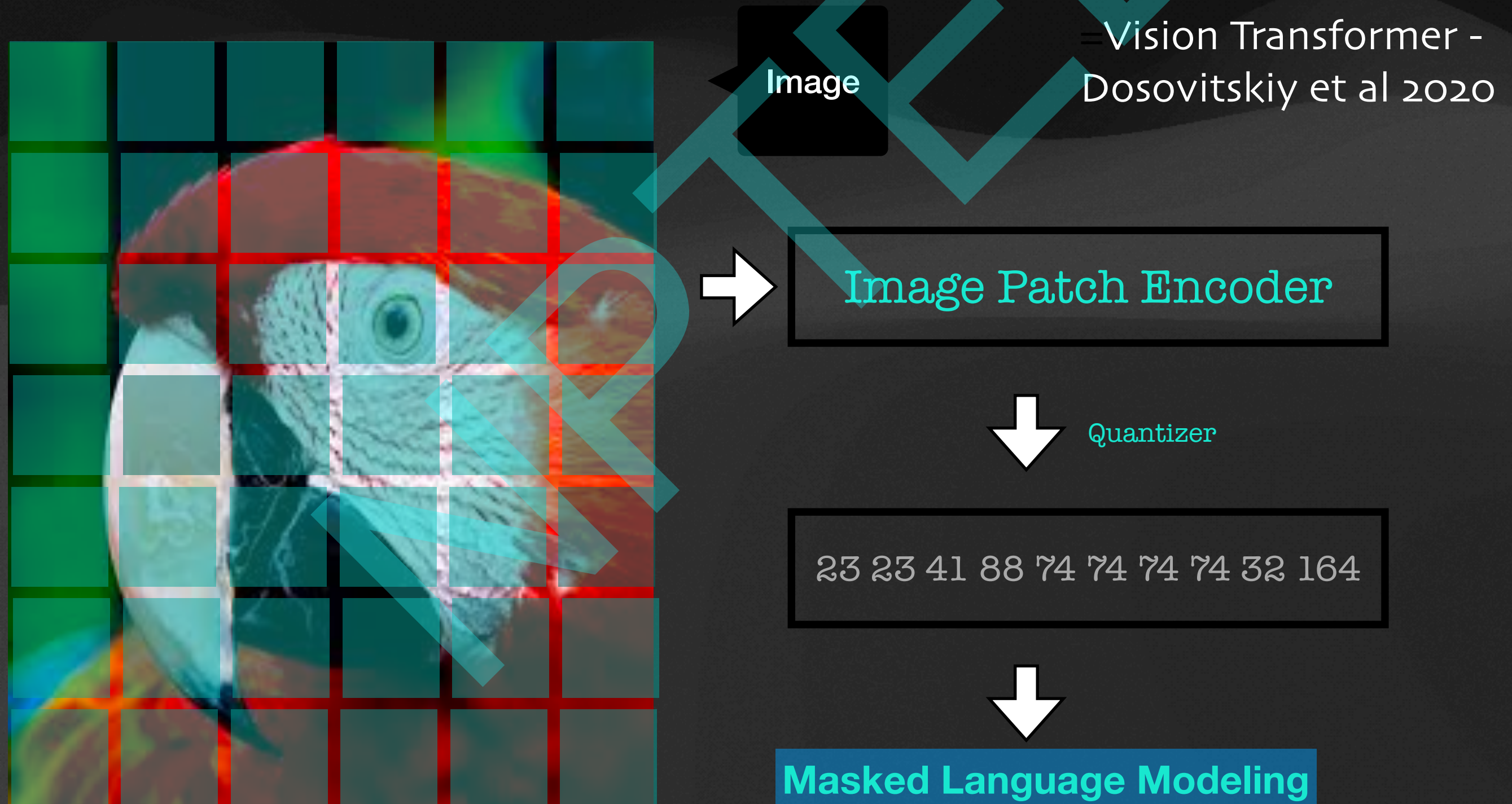
23 23 41 88 74 74 74 74 32 164 164 164 114 114 114 114 88

Masked Language Modeling

HuBERT [Hsu et al 2021]

Beyond text

- ✦ Images can also be converted to a sequence



Sequence data as a universal model

- ✦ Combine the data from all modalities to learn the model - Vocabulary is the union of tokens from all modalities



Towards large models

- ✓ Extending the task of self-supervision
- ✓ Mine lots of text, audio and visual data (and others)
 - * Crawled from the web, as well as, from other resources.
- ✓ Design the model with large capacity (Millions —> Billions of parameters)
- ✓ Pre-train the model
 - * With MLM, next-token prediction and similar style of losses
 - * High resource of computations.
- ✓ Final trained model can be fine-tuned for supervised tasks
 - * Load the parameters as initialization and perform supervised learning.

Evolution of Transformer models

✓ Initial efforts

* Encoder-only models

- ◎ Encoder is bidirectional attention
- ◎ Trained with masked language modeling (MLM) loss for self-supervision followed by task-specific fine-tuning
- ◎ Useful to create generic purpose rich representations of data
 - Suitable for several downstream applications
 - Reduces the dependency of labeled data
- ◎ Popular examples - BERT (Bidirectional Encoder Representations of Transformers) for text [Devlin 2019], HuBERT for speech [Hsu 2021] and ViT for images [Dosovitskiy, 2020]

Evolution of Transformer models

✓ Subsequent efforts

* Encoder-decoder models

- Trained with sequence-to-sequence loss and are often supervised with labels.
- Encoder is bidirectional while decoder is auto-regressive (causal)
- Useful to specific tasks like text-translation, speech-recognition, and image-captioning
- Popular examples - T5 for text [Raffel 2019], Whisper for speech [Radford 2022]

Evolution of Transformer models

✓ Recent efforts

* Decoder only models

- Uses causal attention in an auto-regressive setting
- Trained with next token prediction losses in a self-supervised fashion.
- Useful as a general purpose model for wide-range of tasks.
 - Allows free form generation
- Popular examples - most of the LLMs - GPT series [Radford 2018, ...], Gemini [Team 2023, ...], LLaMa [Touvron 2023, ...], etc

What has revolutionized AI in the last decade

Transformers

Self-supervision

Scale → LLMs

Thank You

NPTEL